

Gain-Scheduled Conductance Programming

Alpha-Beta Initialization and Adaptive Scaling in Memristor Programming

1. Problem Context

In memristor conductance programming, the objective is to drive the device conductance G from an initial value G_0 to a desired target G_{target} using electrical pulses. The system is highly nonlinear, device-dependent, and asymmetric for SET and RESET operations. Therefore, fixed pulse parameters or fixed gains often lead to either:

- Overshoot at low target conductance
- Extremely slow convergence at high target conductance
- Oscillations around the target

To address this, the algorithm implements a **gain-scheduled adaptive scaling strategy** using two parameters:

- α : relative-error gain
- β : absolute-error gain

The effective scaling factor controlling the pulse strength is

$$K = \alpha e_r + \beta e_a$$

where e_r and e_a are relative and absolute error terms.

2. Definitions and Normalization

Let:

- Initial conductance: $G_0 = 5 \mu\text{S}$
- Maximum conductance: $G_{\text{max}} = 250 \mu\text{S}$
- Target conductance: $G_t \in \{60, 100, 150, 200, 250\} \mu\text{S}$

Normalized quantities:

$$g_t = \frac{G_t}{G_{\text{max}}}$$

Error terms:

- Relative error (SET case):

$$e_r = \frac{G_t - G}{G_t}$$

- Absolute error:

$$e_a = \frac{|G_t - G|}{G_{\max}}$$

These definitions are asymmetric and direction-aware, ensuring correct behavior for both SET and RESET.

3. Why Initialization Matters

At the first programming attempt, there is no historical information about how the device will respond. Poor initialization of gains leads to:

- Large overshoot (if gains too large)
- Wasted attempts (if gains too small)

Therefore, the gains α and β must be **initialized as a function of the target conductance**, not chosen arbitrarily.

4. Desired Scaling Factor K_{des}

Instead of guessing α directly, the algorithm first defines a **desired initial scaling factor** K_{des} .

This represents the intended aggressiveness of the first programming step.

Linear Gain Scheduling

For SET operations:

$$K_{\min} = 0.1, \quad K_{\max} = 2.0$$

Target mapping range:

$$G_{\min} = 40 \mu\text{S}, \quad G_{\max}^{\text{lin}} = 250 \mu\text{S}$$

Linear interpolation:

$$t = \frac{\min(\max(G_t, G_{\min}), G_{\max}^{lin}) - G_{\min}}{G_{\max}^{lin} - G_{\min}}$$

$$K_{\text{des}} = K_{\min} + (K_{\max} - K_{\min})t$$

This ensures:

- Small targets $\rightarrow$ gentle programming
- Large targets $\rightarrow$ aggressive programming

5. Solving for α

The scaling law is defined as:

$$K = \alpha(e_r + g_t e_a)$$

At initialization, $K = K_{\text{des}}$, therefore:

$$\alpha = \frac{K_{\text{des}}}{e_r + g_t e_a}$$

Safety limits are applied:

$$0.1 \leq \alpha \leq 3.0$$

6. Definition of β

The absolute-error gain is defined as:

$$\beta = \alpha g_t$$

This choice has deep physical meaning:

- For small targets, g_t is small $\rightarrow \beta$ is small $\rightarrow$ no runaway
- For large targets, g_t is large $\rightarrow \beta$ is large $\rightarrow$ faster escape from saturation
- Near target, $e_a \rightarrow 0 \rightarrow \beta$ naturally vanishes

Safety limit:

$$0 \leq \beta \leq 3.0$$

7. Initialization Results (Numerical Table)

Target (μS)	g_t	K_{des}	α	β
60	0.24	0.28	0.28	0.07
100	0.40	0.46	0.45	0.18
150	0.60	0.73	0.62	0.37
200	0.80	1.18	0.77	0.62
250	1.00	2.00	1.01	1.01

8. Adaptive Update During Programming

At each attempt:

$$K = \alpha e_r + \beta e_a$$

Stuck Detection

If

$$|G_k - G_{k-1}| < 0.1 G_k$$

then the device is considered stuck.

Adaptive Gain Increase

$$\alpha \leftarrow \alpha(1 + \gamma)$$

$$\beta \leftarrow \beta(1 + 0.5\gamma)$$

where

$$\gamma = 0.05 + 0.20 \frac{\text{attempt}}{\text{max attempts}}$$

This ensures slow, controlled escape from stagnation.

9. Emergent Behavior

Without any explicit scheduling:

- K naturally decreases as $G \rightarrow G_t$

- Overshoot probability is minimized
- Low and high target conductance are handled by the same control law

This is **gain scheduling**, not heuristic tuning.

10. Key Insight (Summary)

The desired scaling factor K_{des} encodes *how hard* to push, while α and β encode *how to push* based on where the device currently is.

This separation is what makes the algorithm robust, scalable, and physically meaningful.

11. Final Remarks

This approach is superior to fixed-gain PI/PID control because:

- The system is nonlinear and asymmetric
- The device physics change across the conductance range
- The controller adapts naturally without oscillations

The method is suitable for experimental memristor platforms and scalable to multi-device arrays.